SIMATS ENGINEERING

DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 1 – CO1

Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System

Course Outcome CO1: Algorithm fundamentals, efficiency analysis, asymptotic analysis, mathematical analysis, recurrence analysis, Master Theorem and substitution

Student Name: Thanigaivel S

Register Number: 192565006

Total Marks: 50

1. Introduction and Problem Setting

A university keeps an academic database in which every student record is identified by a unique Registration Number. Faculty and administrative staff repeatedly ask two questions: does a record with a given registration number exist, and if so, what does it contain? The database may hold 1,000, 10,000, 100,000 or more than 1,000,000 records, so the search method must stay practical as the data grows by three orders of magnitude. Any preprocessing, such as sorting or building an index, must be counted in the cost.

This report follows the twelve tasks of the assignment. Each task is answered with design reasoning, mathematical derivation and measured evidence from my own implementation. The measurements were produced by the programs listed in Appendix B, using one fixed random seed so that every algorithm is tested on identical data.

Notation and assumptions

- **n** is the number of records and **q** is the number of searches performed on a given database. **key** is the registration number being searched for.
- Registration numbers are unique 9-digit integers. They are fully ordered, so they can be compared, sorted and hashed.
- Finding the key is treated as retrieving the record. The record itself is reached through a pointer or a parallel array, which adds only constant work once the key is located.
- The records fit in main memory. Disk access is outside the scope of the analysis, although Task 12 comments on it.
- One **key comparison** is the unit of work for all three algorithms, which keeps their costs directly comparable. Execution time is reported as supporting evidence only.

2. Task 1 – Design the Algorithm

I selected three algorithms that differ in the property of the data they exploit. The first exploits nothing, the second exploits ordering, and the third exploits the key's numeric value to compute a location. Their design strategies are therefore different, not merely different implementations of one idea.

2.1 Approach A – Sequential Search (iterative, brute force)

Basic idea. Examine the records one at a time from the first position, comparing each stored registration number with the key. Stop as soon as a match is found. If the end of the array is reached, the key is absent.

Preprocessing. None. The records can stay in whatever order they were entered, and new records are simply appended.

Why it was selected. It is the natural baseline against which every other method must be judged. It also works on unsorted data and has no preprocessing cost, so it is the best choice when only one or a few searches are performed.

```
ALGORITHM SequentialSearch(A, n, key)
INPUT:
    A – array of n registration numbers in any order
    n – number of records
    key – registration number to locate
OUTPUT:
    index i such that A[i] = key, or NOT_FOUND
BEGIN
    FOR i ← 0 TO n - 1 DO
        IF A[i] = key THEN
            RETURN i
        END IF
    END FOR
    RETURN NOT_FOUND
END
```

2.2 Approach B – Binary Search (recursive, decrease-and-conquer)

Basic idea. If the registration numbers are sorted, one comparison with the middle element tells us which half of the array can still contain the key. The other half is discarded without being examined. The same question is then asked of the remaining half, and the search space shrinks by a factor of two at every step. This is a decrease-and-conquer strategy (decrease by half), and the sorting step is an instance of presorting, a form of transform-and-conquer.

Preprocessing. The records must be sorted in ascending order of registration number. I use merge sort, which guarantees Θ(n log n) comparisons whatever the input order. Sorting is performed once, and every later search benefits from it. This cost is included in all later analysis.

Why it was selected. Registration numbers are unique and comparable, which is exactly the situation in which one comparison can eliminate half of the remaining candidates. The method needs no extra memory beyond the sorted array and also supports ordered operations such as range queries.

```
ALGORITHM BinarySearch(A, low, high, key)
INPUT:
    A – array of registration numbers sorted in ascending order
    low, high – boundaries of the current search space
    key – registration number to locate
OUTPUT:
    index i such that A[i] = key, or NOT_FOUND
BEGIN
    IF low > high THEN
        RETURN NOT_FOUND
    END IF
    mid ← FLOOR((low + high) / 2)
    IF A[mid] = key THEN
        RETURN mid
    ELSE IF key < A[mid] THEN
        RETURN BinarySearch(A, low, mid - 1, key)
    ELSE
        RETURN BinarySearch(A, mid + 1, high, key)
    END IF
END
```

The initial call is BinarySearch(A, 0, n - 1, key). The preprocessing step is merge sort:

```
ALGORITHM MergeSort(A, low, high)
INPUT:
    A – array of registration numbers
    low, high – boundaries of the part to be sorted
OUTPUT:
    A[low..high] arranged in ascending order
BEGIN
    IF low ≥ high THEN
        RETURN
    END IF
    mid ← FLOOR((low + high) / 2)
    MergeSort(A, low, mid)
    MergeSort(A, mid + 1, high)
    i ← low;  j ← mid + 1;  k ← 0;  INITIALISE empty array Temp
    WHILE i ≤ mid AND j ≤ high DO
        IF A[i] ≤ A[j] THEN  Temp[k] ← A[i];  i ← i + 1
        ELSE  Temp[k] ← A[j];  j ← j + 1
        END IF
        k ← k + 1
    END WHILE
    WHILE i ≤ mid DO  Temp[k] ← A[i];  i ← i + 1;  k ← k + 1  END WHILE
    WHILE j ≤ high DO  Temp[k] ← A[j];  j ← j + 1;  k ← k + 1  END WHILE
    FOR t ← 0 TO k - 1 DO  A[low + t] ← Temp[t]  END FOR
END
```

A loop-based version of binary search is the same algorithm and would not count as a separate approach. I implemented one only to cross-check the recursive version.

2.3 Approach C – Hash Table with Separate Chaining (iterative, transform-and-conquer)

Basic idea. Instead of searching for the key, compute where it must be. A hash function maps the registration number to a bucket number, and each bucket holds a

short list of the keys that map to it. A search computes the bucket number and scans only that short list. This is a representation change combined with a space–time trade-off: extra memory buys a much shorter search.

Preprocessing. A table of m buckets is built by inserting every key. I choose m as the smallest prime not less than n / 0.75, so the load factor is α = n / m ≈ 0.75. The hash function is h(key) = key mod m. A prime modulus spreads keys more evenly than a power of two would.

Why it was selected. Searching by exact match does not need any ordering. A method that goes straight to the right place can in principle make the number of comparisons independent of n, which makes it the natural contender for very large databases.

```
ALGORITHM BuildHashTable(A, n)
INPUT:
    A – array of n distinct registration numbers
    n – number of records
OUTPUT:
    Table – array of m buckets, each an ordered list of keys;  m – number of
    buckets
BEGIN
    m ← smallest prime number ≥ n / 0.75
    FOR b ← 0 TO m - 1 DO  Table[b] ← empty list  END FOR
    FOR i ← 0 TO n - 1 DO
        b ← A[i] MOD m
        APPEND A[i] to the end of Table[b]
    END FOR
    RETURN Table, m
END
```

```
ALGORITHM HashSearch(Table, m, key)
INPUT:
    Table, m – structure produced by BuildHashTable
    key – registration number to locate
OUTPUT:
    TRUE if key is stored, otherwise FALSE
BEGIN
    b ← key MOD m
    FOR each x in Table[b], in stored order DO
        IF x = key THEN
            RETURN TRUE
        END IF
    END FOR
    RETURN FALSE
END
```

2.4 Summary of the three designs

Table 1: Distinct strategies selected for investigation

	A: Sequential	B: Binary	C: Hash table
Design strategy	Brute force	Decrease-and-conquer + presorting	Transform-and-conquer (hashing)
Style	Iterative	Recursive	Iterative
Data property used	None	Total ordering of keys	Numeric value of key
Preprocessing	None	Merge sort	Build table (n insertions)
Extra memory	None	None (sorted array)	Buckets, about α = 0.75

3. Task 2 – Identify the Basic Operation

The basic operation of an algorithm is the operation in the innermost, most frequently repeated part, whose count grows with n and therefore dominates the running time. Everything else in the algorithm, such as updating a loop index or computing a

midpoint, happens a number of times proportional to this operation and only changes the constant factor.

Approach A – Sequential search

The basic operation is the **comparison A[i] = key**. It occurs inside the loop body, once per iteration, and the loop is the only repeated structure in the algorithm. Incrementing i and testing the loop bound happen exactly as often, so they differ by a constant factor. It is appropriate to count comparisons because the running time is essentially the number of iterations, and that number depends on where the key sits. As n grows, the number of comparisons grows in direct proportion: from 1 when the key is first to n when it is last or absent.

Approach B – Binary search

The basic operation is the **comparison of the key with A[mid]**. It occurs once in every recursive call, immediately after the midpoint is computed. Each call either finishes the search or hands a search space of at most half the size to one further call, so the number of comparisons equals the number of calls made. This count changes only by one every time n doubles, because doubling n adds one more halving step. For the preprocessing, the basic operation is the **comparison A[i] ≤ A[j]** in the merge step of merge sort. Each comparison places one element into its final position in the merged run, and the number of such comparisons grows slightly faster than n.

Approach C – Hash table

For searching, the basic operation is the **comparison x = key** inside the bucket scan. Computing the bucket number is a single arithmetic step performed once per search, and it does not depend on n. The cost that varies is how many keys must be compared in the chosen bucket. That number depends on the load factor $\alpha = n / m$ and not on n alone, so when the table grows in proportion to n the comparison count stays roughly constant. For building the table, the basic operation is the **insertion of one key** (one hash computation and one append), performed n times.

Table 2: Basic operations

Algorithm	Basic operation	Where it occurs	Executions as n grows
A	A[i] = key	Loop body	Proportional to n

Algorithm	Basic operation	Where it occurs	Executions as n grows
B (search)	key vs A[mid]	Each recursive call	+1 per doubling of n
B (sort)	A[i] ≤ A[j]	Merge step	About n $\log_2$ n
C (search)	x = key in chain	Bucket scan	About 1 + α/2, independent of n
C (build)	Insert one key	Build loop	Exactly n

4. Task 3 – Derive the Time-Complexity Function Mathematically

For every algorithm the input size is n, the number of records, and the cost is the number of executions of the basic operation identified in Task 2. For the hash table, α = n / m denotes the load factor.

4.1 Sequential search

The loop examines positions 0, 1, …, n − 1 in order. If the key is at position i (counting from 1), the loop executes the comparison exactly i times and stops. If the key is absent, every position is examined:

$$C_{unsuccessful}(n) = \Sigma_{i=1}^{n} 1 = n$$

For a successful search, assume the key is equally likely to be at each of the n positions. The expected number of comparisons is then

$$C_{success}(n) = (1/n)\ \Sigma_{i=1}^{n} i = (1/n) \cdot n(n + 1)/2 = (n + 1)/2$$

If a fraction p of the searches are successful, the average becomes p(n + 1)/2 + (1 − p)n, which is a linear function of n for every p. There is no preprocessing, so the cost of q searches is q · Θ(n), that is $\mathbf{T_A(n, q) = \Theta(q \cdot n)}$.

4.2 Binary search

Each call makes one comparison and passes a search space of at most ⌊n/2⌋ elements to the next call. After k comparisons at most n / 2^k elements remain, and the search must stop when that number reaches zero. The largest possible number of comparisons is therefore the number of times n can be halved before nothing is left:

$$C_{worst}(n) = \lfloor \log_2 n \rfloor + 1$$

This is proved formally by solving the recurrence in Task 7. For the average successful case, take $n = 2^k - 1$, so that the comparison tree is a perfect binary tree with k levels. Level i contains 2^{i-1} keys, and each of them is found after exactly i comparisons:

$$C_{success}(n) = (1/n)\ \Sigma_{i=1}^{k}\ i \cdot 2^{i-1} = ((k-1)2^k + 1) / (2^k - 1) = (k-1) + k/n$$

With $k = \log_2(n+1)$, this gives $\mathbf{C_{success}(n) \approx \log_2(n+1) - 1}$. For an unsuccessful search, the search ends at one of n + 1 gaps between keys, and all of them lie at depth $\lfloor \log_2 n \rfloor$ or $\lfloor \log_2 n \rfloor + 1$, so the cost is about $\log_2(n+1)$. For n = 1,000,000 the formulas give 18.93 comparisons for a successful search and about 19.93 for an unsuccessful one.

The preprocessing is merge sort. Merging two sorted runs of total length L needs at most L − 1 comparisons, so the number of comparisons satisfies $M(n) = M(\lceil n/2 \rceil) + M(\lfloor n/2 \rfloor) + (n-1)$ in the worst case, whose solution (Task 7) is about $n \log_2 n - n + 1$. For random input the average is close to $n \log_2 n - 1.25n$. The overall cost of preparing the data and then performing q searches is

$$T_B(n, q) \approx n \log_2 n + q \log_2 n = \Theta((n+q) \log n)$$

4.3 Hash table with separate chaining

Assume simple uniform hashing: each key is equally likely to fall into any of the m buckets. The expected length of a chain is $\alpha = n / m$.

Unsuccessful search. The bucket is chosen and every key in the chain is compared with the key, so the expected number of comparisons is **α**.

Successful search. Suppose the searched key was the i-th one inserted. The keys that sit ahead of it in its chain are those inserted earlier that landed in the same bucket. Each of the i − 1 earlier keys does so with probability 1/m, so the expected number of comparisons is 1 + (i − 1)/m. Averaging over all n positions of the searched key:

$$C_{success} = 1 + (1/n)\ \Sigma_{i=1}^{n}\ (i-1)/m = 1 + (n-1)/(2m) \approx 1 + \alpha/2$$

With $\alpha \approx 0.75$, this predicts 1.375 comparisons for a successful search and 0.75 for an unsuccessful one, whatever the value of n. Building the table inserts n keys at constant cost each, so the preprocessing cost is exactly n insertions, Θ(n). The

overall cost is therefore $\mathbf{T_C(n, q) = \Theta(n + q)}$ in expectation. If the hash function is poor, every key could land in a single bucket, and a search would then cost up to n comparisons. This worst case is demonstrated in Task 4.

4.4 Preprocessing and searching together

Table 3: Derived cost functions

Algorithm	Preprocessing	One search (average)	q searches in total
A: Sequential	0	Θ(n)	Θ(q · n)
B: Binary	Θ(n log n)	Θ(log n)	Θ(n log n + q log n)
C: Hash table	Θ(n)	Θ(1) expected	Θ(n + q) expected

These expressions show that sequential search is cheapest when q is very small, because it pays nothing in advance. The two preprocessing-based methods pay a fixed cost once and then reduce the per-search cost. Task 11 calculates exactly how many searches are needed before that investment pays off.

5. Task 4 – Best, Average and Worst-Case Behaviour

5.1 Sequential search

Best case. The key is stored in the first position, so the first comparison succeeds and only one comparison is made. This is independent of n.

Worst case. The key is either in the last position or not in the database at all. The algorithm has no information about where the key might be, so it cannot rule out a position without looking at it. An absent key is the more expensive of the two situations in practice, because proving absence requires inspecting all n records.

Average case. If the key is equally likely to be anywhere, then on average half the records are examined before it is found, which gives (n + 1)/2 comparisons. A search for an absent key always costs n, so a workload with many mistyped registration numbers is more expensive than one with only valid numbers.

5.2 Binary search

Best case. The key sits exactly at the middle of the sorted array, so the first probe finds it and one comparison is made.

Worst case. The key is found only in the deepest call, or it is absent. The search space is halved on every call until it is empty, which takes $\lfloor \log_2 n \rfloor + 1$ comparisons.

Average case. Half of all keys lie at the deepest level of the comparison tree, a quarter at the level above, and so on. Almost every key therefore needs close to the maximum number of probes, and the average of about $\log_2 n - 1$ is only one probe below the worst case. The best case of a single probe is rare, because only one key can be at the first midpoint. Unlike sequential search, the result does not depend on whether the key is present or absent.

5.3 Hash table

Best case. The key is the first one in its chain, so one comparison is enough. A search for an absent key whose bucket is empty needs no comparison at all.

Average case. With a good hash function the chains are short and nearly uniform, with an expected length of α. A successful search then needs about $1 + \alpha/2$ comparisons and an unsuccessful one about α.

Worst case. All keys fall into the same bucket. The table has then degenerated into one long list and the search is exactly sequential search, requiring up to n comparisons. This can happen if the keys have structure that matches the hash function. For example, if the registration numbers are all multiples of m, then key mod m is always 0. I constructed such an input to test this behaviour, as shown in Table 4.

Table 4: Measured best, average and worst-case comparisons

Algorithm	Best case	Average case (n = 1,000,000)	Worst case (n = 1,000,000)
A: Sequential	1	472,548 successful; 1,000,000 unsuccessful	1,000,000
B: Binary	1	18.95 successful; 19.95 unsuccessful	20
C: Hash (random keys)	1 (0 if bucket empty)	1.376 successful; 0.748 unsuccessful	7 (longest chain)
C: Hash (all keys collide)	1	5,000 successful (n = 10,000)	10,000 (n = 10,000)

The all-collision input of 10,000 keys produced a single chain of length 10,000, and the search for its last element took exactly 10,000 comparisons, confirming that the worst case of hashing is Θ(n). On the random registration numbers, however, the longest chain among the one million keys had only 7 entries. The worst case is therefore a property of the input structure and not of the input size.

6. Task 5 – Asymptotic Analysis

Asymptotic notation describes growth for large n while ignoring constant factors. I use O for an upper bound, Ω for a lower bound, and Θ where both hold.

Table 5: Asymptotic classification of one search

Algorithm	Best	Average	Worst	Preprocessing
A: Sequential	Ω(1)	Θ(n)	Θ(n)	None
B: Binary	Ω(1)	Θ(log n)	Θ(log n)	Θ(n log n)
C: Hash table	Ω(1)	Θ(1) expected	Θ(n)	Θ(n)

Justification with explicit constants

- **Sequential, worst case.** $C(n) = n \le 1 \cdot n$ for all $n \ge 1$ and $n \ge 1 \cdot n$, so $C(n) = \Theta(n)$ with $c_1 = c_2 = 1$ and $n_0 = 1$. The average $(n + 1)/2$ satisfies $n/2 \le (n + 1)/2 \le n$ for $n \ge 1$, so it is also Θ(n).
- **Binary, worst case.** $\lfloor \log_2 n \rfloor + 1 \le 2 \log_2 n$ for $n \ge 2$, giving O(log n). Since $\lfloor \log_2 n \rfloor + 1 > \log_2 n$, it is also Ω(log n), and therefore Θ(log n), with $n_0 = 2$.
- **Hash table, expected case.** $1 + \alpha/2$ is at most 1.375 when $\alpha \le 0.75$, so the expected cost is bounded by a constant independent of n, i.e. Θ(1). The best case Ω(1) is trivial because at least the hash computation is performed.

The three growth rates are strictly ordered, $1 < \log n < n < n \log n$. For example, $\lim_{n\to\infty} (\log_2 n)/n = 0$ by L'Hôpital's rule, which means log n grows more slowly than n. The same ordering applies to the preprocessing costs: n log n for sorting and n for hashing.

Growth from 1,000 to 1,000,000 records

Table 6: Effect of a thousand-fold increase in database size

Algorithm	Predicted growth in comparisons	Measured growth in comparisons	Measured growth in time

Algorithm	Predicted growth in comparisons	Measured growth in comparisons	Measured growth in time
A: Sequential	× 1000	× 941	× 1,252
B: Binary	× 2.11	× 2.11	× 3.6
C: Hash table	× 1.00	× 1.01	× 5.9

In the context of the student-record system, this means that a database a thousand times larger makes sequential search about a thousand times slower. A lookup that was instantaneous for a departmental database becomes a noticeable delay for a university-wide one. Binary search needs only about 10 extra comparisons, roughly twice as many as before, because the number of halvings grows by one for each doubling of the data. Hash lookup does not need more comparisons at all. The measured time of the hash lookup nevertheless grew, for a reason discussed in Task 10.

7. Task 6 – Formulate the Recurrence Relation

The recursive approach is binary search (Approach B). The recurrence is derived directly from the pseudocode in Section 2.2.

- **Size of the original problem.** n = high − low + 1, the number of elements still in the search space.
- **Size of the subproblem.** After comparing with A[mid], the left part contains $\lfloor (n-1)/2 \rfloor$ elements and the right part $\lceil (n-1)/2 \rceil = \lfloor n/2 \rfloor$ elements. In the worst case the larger part is searched, so the subproblem has size $\lfloor n/2 \rfloor$.
- **Number of recursive calls.** Exactly one. The if–else structure sends the search to the left half or to the right half but never to both, so a = 1.
- **Work outside the recursive call.** Computing mid, one comparison with A[mid], and one comparison of low with high. This is a constant amount of work, f(n) = Θ(1), and it contains exactly one basic operation.
- **Base case.** When n = 0 (low > high) the function returns NOT_FOUND immediately without making any key comparison, so T(0) = 0. If n = 1, one comparison is made before the call on the empty remainder, so T(1) = 1.

Counting key comparisons, the worst-case recurrence is therefore:

$$T(n) = T(\lfloor n/2 \rfloor) + 1 \quad \text{for } n \geq 1, \qquad T(0) = 0$$

The term $T(\lfloor n/2 \rfloor)$ represents the single recursive call on the half that remains. The term +1 represents the comparison made in the current call before that recursive call. If the key is found early the recursion stops sooner, so this recurrence describes the worst case, which is also the case for an unsuccessful search.

Recurrence for the preprocessing. Merge sort is also recursive, so its comparison count can be derived the same way. The problem of size n is divided into two subproblems of sizes $\lceil n/2 \rceil$ and $\lfloor n/2 \rfloor$ (two recursive calls), and the merge step performs at most $n - 1$ comparisons outside the calls. The base case is a single element, which needs no comparison:

$$M(n) = M(\lceil n/2 \rceil) + M(\lfloor n/2 \rfloor) + (n - 1), \qquad M(1) = 0$$

8. Task 7 – Solve the Recurrence Relation

8.1 Binary search: $T(n) = T(\lfloor n/2 \rfloor) + 1$, $T(0) = 0$

Choice of methods. The recurrence has one subproblem of half size and a constant amount of other work, so repeated back-substitution (the iteration method) works well. I use it first for $n = 2^k$ to see the pattern, then confirm it for all n by induction (the substitution method), and finally cross-check with the Master Theorem.

Iteration. Let $n = 2^k$. Expanding the recurrence repeatedly gives:

$$T(2^k) = T(2^{k-1}) + 1 = T(2^{k-2}) + 2 = T(2^{k-3}) + 3 = \ldots = T(2^0) + k$$

Since $T(1) = T(0) + 1 = 1$, we get $T(2^k) = k + 1$, that is $T(n) = \log_2 n + 1$ for powers of two.

Substitution (induction) for every n. Claim: $T(n) = \lfloor \log_2 n \rfloor + 1$ for all $n \geq 1$.

- Base case: $T(1) = 1 = \lfloor \log_2 1 \rfloor + 1 = 0 + 1$.
- Inductive step: let $n \geq 2$ and assume the claim for all smaller values. Then $T(n) = T(\lfloor n/2 \rfloor) + 1 = \lfloor \log_2 \lfloor n/2 \rfloor \rfloor + 1 + 1$.
- If $2^j \leq n < 2^{j+1}$, then $2^{j-1} \leq \lfloor n/2 \rfloor < 2^j$, so $\lfloor \log_2 \lfloor n/2 \rfloor \rfloor = j - 1 = \lfloor \log_2 n \rfloor - 1$.
- Therefore $T(n) = (\lfloor \log_2 n \rfloor - 1) + 2 = \lfloor \log_2 n \rfloor + 1$, which completes the proof.

Master Theorem check. The recurrence has the form $T(n) = aT(n/b) + f(n)$ with $a = 1$, $b = 2$ and $f(n) = \Theta(1) = \Theta(n^0)$. The critical exponent is $\log_b a = \log_2 1 = 0$, so $n^{\log_b a} = n^0 = 1$. Since $f(n) = \Theta(n^{\log_b a})$, Case 2 of the Master Theorem applies, and $T(n) =$

$\Theta(n^{\log_b a} \cdot \log n) = \mathbf{\Theta(\log n)}$. The theorem applies because $a \geq 1$, $b > 1$ and $f(n)$ is a polynomial-type function; the floor in the subproblem size does not change the asymptotic result.

Interpretation. The exact solution $\lfloor \log_2 n \rfloor + 1$ predicts 10, 14, 17 and 20 comparisons in the worst case for 1,000, 10,000, 100,000 and 1,000,000 records. These values match the largest number of comparisons observed in the experiments (Task 9). The recursion depth equals the number of comparisons, so the stack space is $\Theta(\log n)$, which is only 20 frames for a million records. The recursive form is therefore perfectly safe for this application. Figure 1 shows how quickly the search space collapses.

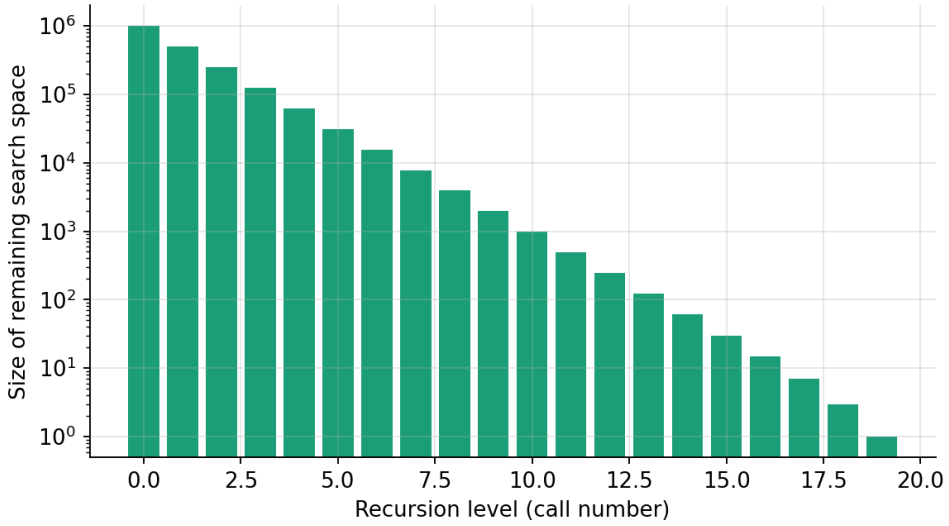


Figure 1: Size of the remaining search space at each recursive call of binary search for n = 1,000,000. It reaches zero after 20 calls.

8.2 Merge sort (preprocessing): M(n) = 2M(n/2) + n − 1

Taking $n = 2^k$ and expanding:

$$M(n) = 2M(n/2) + n - 1 = 4M(n/4) + 2n - 3 = \ldots = 2^i\, M(n/2^i) + i \cdot n - (2^i - 1)$$

Setting $i = k = \log_2 n$, where $M(1) = 0$, gives $M(n) = n \log_2 n - (n - 1) = \mathbf{n \log_2 n - n + 1}$. The Master Theorem agrees: $a = 2$, $b = 2$, $f(n) = n = \Theta(n^{\log_2 2})$, so Case 2 applies and $M(n) = \Theta(n \log n)$. This is the preprocessing cost that Task 3 added to the search cost, and it confirms that sorting a million records needs on the order of twenty million comparisons.

9. Task 8 – Implement and Test the Algorithms

9.1 Implementation

All approaches are implemented in Python 3.12 (CPython) in the module search_lib.py, listed in Appendix B. Each algorithm exists in an instrumented form that returns the number of key comparisons, and in a plain form with identical logic and no counters, used only for timing. The code corresponds line by line to the pseudocode of Task 1.

- **Sequential search** is a for-loop over the array (iterative).
- **Binary search** is the recursive function of the pseudocode. A loop version is included only to confirm that both give the same answer.
- **Hash table** uses separate chaining with a prime number of buckets, the hash function key mod m, and a load factor of 0.75.
- **Preprocessing** uses a bottom-up merge sort that counts every key comparison, together with the table-building loop that counts every insertion.

9.2 Datasets and testing procedure

- **Sizes.** n = 1,000, 10,000, 100,000 and 1,000,000 records.
- **Data.** Unique 9-digit registration numbers drawn at random (seed 2024 plus n) and stored in random order. Exactly the same array is given to all three algorithms, and the sorted array and hash table are built from it.
- **Successful searches.** 20,000 randomly chosen existing keys for binary search and hashing. Because one sequential search of a million records costs hundreds of thousands of comparisons, sequential search uses 200 random keys for n ≤ 100,000 and 60 for n = 1,000,000.
- **Unsuccessful searches.** Random 9-digit numbers confirmed not to be in the database: 20,000 for binary search and hashing, and 100 (20 for n = 1,000,000) for sequential search.
- **Designed cases.** The first stored key, the last stored key, the middle of the sorted order, and adversarial hash input.
- **Environment.** A single CPython process on a shared container, so absolute times are not comparable with compiled code. Comparison counts are exact and machine-independent, so they form the primary evidence.

9.3 Correctness

For each of the four sizes, 4,000 queries (2,000 existing and 2,000 absent keys) were checked against Python's set membership as an independent reference. Every algorithm returned the correct answer for all 16,000 of 16,000 checks. For found keys, the returned position was also confirmed to hold the key. Table 7 lists nine small hand-checkable cases that cover the boundary conditions.

Table 7: Boundary and edge-case tests

Test case	Expected	Comparisons (A / B / C)	Result
Empty database	not found	0 / 0 / 0	Pass
Single record, present	found	1 / 1 / 1	Pass
Single record, absent	not found	1 / 1 / 0	Pass
7 records, first stored	found	1 / 3 / 1	Pass
7 records, last stored	found	7 / 3 / 1	Pass
7 records, middle of sorted order	found	6 / 1 / 1	Pass
7 records, smaller than all	not found	7 / 3 / 0	Pass
7 records, larger than all	not found	7 / 3 / 1	Pass
7 records, between two keys	not found	7 / 3 / 1	Pass

These tests verify specific behaviours: an empty database returns not found without error, a single record is handled by all three methods, keys smaller than all records and larger than all records end the binary search correctly at the array boundary, and a key lying between two stored numbers is rejected. The hash table needs no comparison when the bucket is empty, which is why some counts for it are 0.

10. Task 9 – Record Experimental Performance

10.1 Number of key comparisons

Table 8: Average comparisons per successful search

n	Sequential	Binary	Hash table	Binary maximum
1,000	501.95	8.98	1.366	10
10,000	5,212.65	12.37	1.375	14
100,000	47,342.31	15.71	1.363	17
1,000,000	472,548.37	18.95	1.376	20

Table 9: Average comparisons per unsuccessful search

n	Sequential	Binary	Hash table	Hash longest chain
1,000	1,000	9.98	0.735	4
10,000	10,000	13.37	0.742	6
100,000	100,000	16.69	0.746	6
1,000,000	1,000,000	19.95	0.748	7

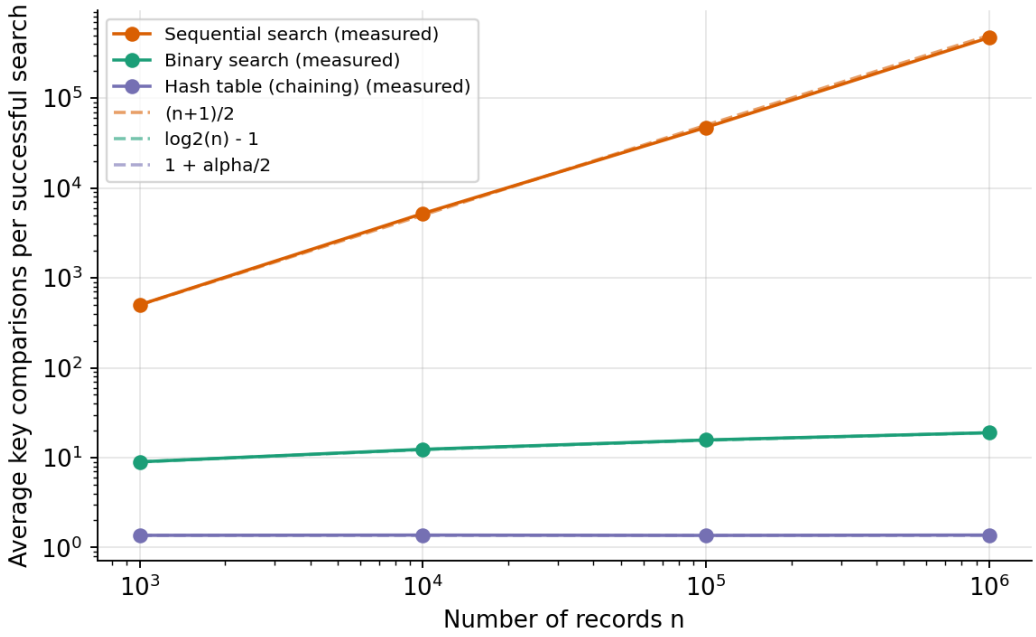


Figure 2: Average comparisons per successful search against database size (both axes logarithmic). Dashed lines show the theoretical predictions.

10.2 Execution time

Table 10: Average time per search in microseconds (successful / unsuccessful)

n	Sequential (µs)	Binary (µs)	Hash table (µs)
1,000	8.21 / 17.3	0.94 / 1.07	0.12 / 0.13
10,000	113.5 / 214.6	1.33 / 1.52	0.19 / 0.11
100,000	1,013 / 2,146	2.02 / 2.09	0.43 / 0.26
1,000,000	10,286 / 22,412	3.36 / 4.09	0.74 / 0.52

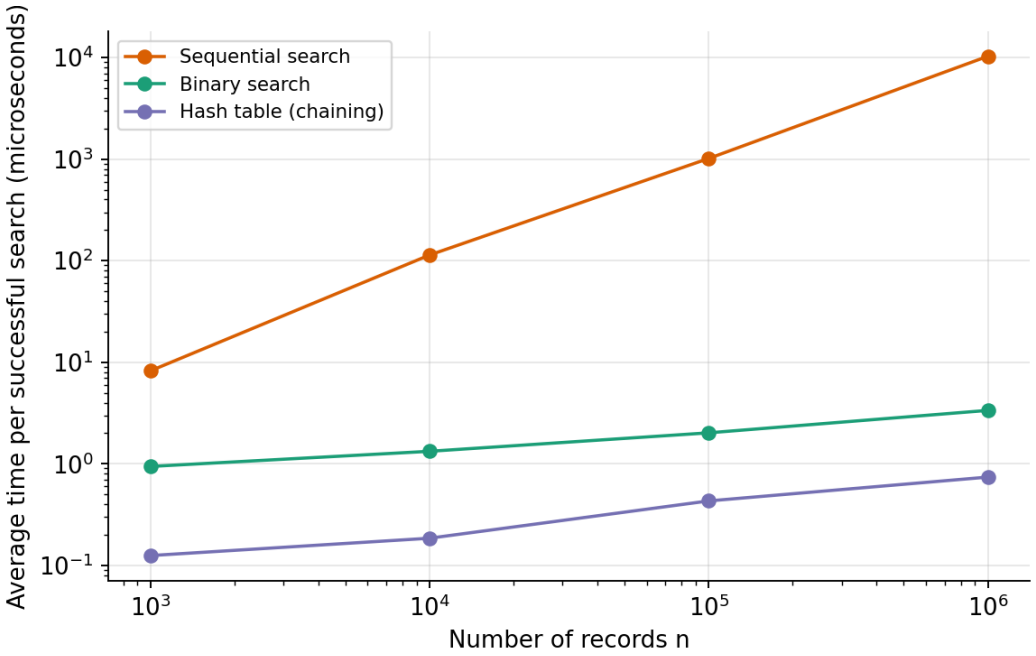


Figure 3: Average time per successful search in microseconds (both axes logarithmic).

10.3 Preprocessing cost and memory

Table 11: Preprocessing cost (counted merge sort and table construction)

n	Sort comparisons	Sort time (s)	Hash insertions	Hash build time (s)
1,000	8,697	0.001	1,000	0.000
10,000	123,755	0.014	10,000	0.004
100,000	1,566,164	0.192	100,000	0.039
1,000,000	18,716,484	3.281	1,000,000	1.676

Table 12: Index memory (pointer arrays only; the key objects are shared by all structures)

n	Sorted array (MB)	Hash table (MB)	Load factor α	Buckets m
1,000	0.01	0.07	0.735	1,361
10,000	0.09	0.69	0.750	13,337
100,000	0.80	6.86	0.750	133,337
1,000,000	8.45	68.65	0.750	1,333,357

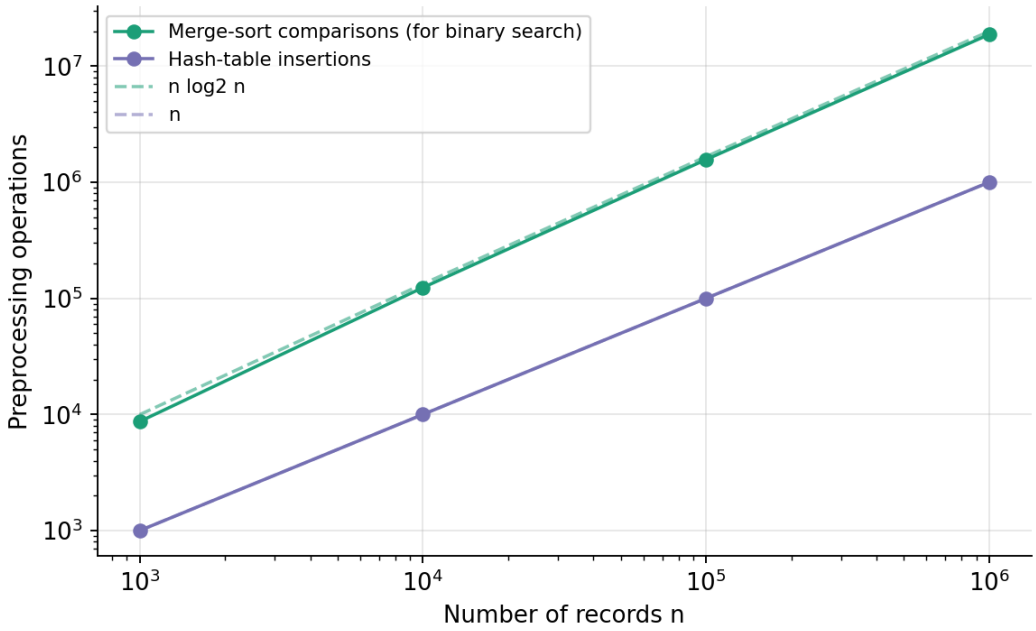


Figure 4: Preprocessing operations against database size. The dashed lines are n $\log_2 n$ and n.

The data show three clearly separated behaviours. Sequential search shows the largest growth in every measure, binary search grows by only a few comparisons per thousand-fold increase in data, and the hash table stays flat. The price paid by the second and third methods is visible in Tables 11 and 12: sorting a million records takes about 18.7 million comparisons, and the hash table needs about eight times as much index memory as the sorted array.

11. Task 10 – Compare Theory with Experimental Behaviour

Table 13: Predicted and measured successful-search comparisons (A: (n+1)/2; B: formula $\log_2(n+1) - 1 + \log_2(n+1)/n$, and exact value from the comparison tree)

n	A theory	A measured	B formula	B exact	B measured
1,000	500.5	502.0	8.98	8.99	8.98
10,000	5,000.5	5,212.6	12.29	12.36	12.37
100,000	50,000.5	47,342.3	15.61	15.69	15.71
1,000,000	500,000.5	472,548.4	18.93	18.95	18.95

Table 14: Predicted and measured comparisons for the hash table

n	C predicted 1+(n−1)/2m	C measured	C predicted α	C measured (unsucc.)

n	C predicted 1+(n−1)/2m	C measured	C predicted α	C measured (unsucc.)
1,000	1.367	1.366	0.735	0.735
10,000	1.375	1.375	0.750	0.742
100,000	1.375	1.363	0.750	0.746
1,000,000	1.375	1.376	0.750	0.748

Does the observed growth agree with the theory?

Binary search and hashing agree closely. For binary search, the measured average differs from the exact value for the comparison tree by at most 0.03 comparison. The simple closed formula of Task 3 is derived for $n = 2^k - 1$ and underestimates by up to 0.08 for the other sizes, because the last level of the tree is then only partly filled and its keys need one extra probe. For the hash table, the measured successful-search average is within 0.012 of the prediction at every size. The maximum number of comparisons for binary search was exactly $\lfloor \log_2 n \rfloor + 1$ at every size (10, 14, 17, 20), as the solution of the recurrence predicts. The preprocessing counts also agree: the measured merge-sort comparisons are between 87% and 94% of $n \log_2 n$, consistent with the known average of about $n \log_2 n - 1.25n$ for random data, and the hash table performed exactly n insertions.

Sequential search shows a sampling effect. At n = 1,000,000 only 60 random keys were used, and the measured average of 472,548 is 5.5% below the predicted 500,000. Positions of random keys are uniformly distributed with standard deviation $n/\sqrt{12}$, so the standard error of a 60-key average is about 7.5% of the mean. The deviation is within one standard error. It is a property of the small sample, not a failure of the formula. The unsuccessful searches cost exactly n at every size, again as predicted.

Which algorithm scales better?

Judged by the number of operations, the order is hash table (constant), then binary search (logarithmic), then sequential search (linear). The difference between the first two is small: at a million records, binary search needs about 19 comparisons where hashing needs about 1.4. Both are separated from sequential search by more than four orders of magnitude.

Differences between prediction and execution, and their causes

- **Time grew faster than comparisons for binary search.** The comparison count grew about 2.1 times between n = 1,000 and 1,000,000, but the time per search grew about 3.6 times. The extra growth comes from the memory hierarchy: a 1,000-element array fits in the processor's fastest cache, whereas a million-element array does not, so each probe is more likely to wait for slower memory. The theory counts operations and assumes each costs the same.
- **The hash table time rose although its comparison count did not.** Time per search grew about 5.9 times. Each lookup jumps to a pseudo-random bucket, which for a table of about 69 MB almost always causes a cache miss. This is the hidden cost of hashing that operation counts do not show.
- **Sequential time grew slightly more than n.** It grew about 1,252 times for a 1,000-fold increase, again because large arrays exceed the cache. The successful/unsuccessful time ratio (2.18 at n = 1,000,000) matches the predicted ratio of about 2, because absent keys scan the whole array and present keys scan half of it on average.
- **Interpreter overhead.** Python adds a large constant to every operation, particularly to recursion. The constants would differ in a compiled language, but the growth rates would not, which is why the comparison counts are the primary evidence.

12. Task 11 – Recommend and Defend the Final Design

12.1 When does the preprocessing pay for itself?

Because sequential search has no preprocessing, the question is how many searches q* are needed before the cheaper searches of the other methods make up for their preprocessing cost. Using the measured preprocessing cost S(n) and the average successful-search costs, the break-even is $q^* = S(n) / (c_A - c_X)$, where $c_A = (n + 1)/2$ is the sequential cost and c_X the cost of the method being compared.

Table 15: Break-even number of searches against sequential search

n	Binary: searches to break even	Hash: searches to break even

n	Binary: searches to break even	Hash: searches to break even
1,000	18	3
10,000	25	3
100,000	32	3
1,000,000	38	3

The result is striking. Even for a million records, sorting is recovered after only about 38 searches, and building a hash table after 3. Any system that answers more than a few dozen lookups should therefore not use sequential search. Figure 5 shows the cumulative cost for n = 1,000,000. The curves for binary search and hashing start higher because of preprocessing, and then become almost flat compared with the sequential line.

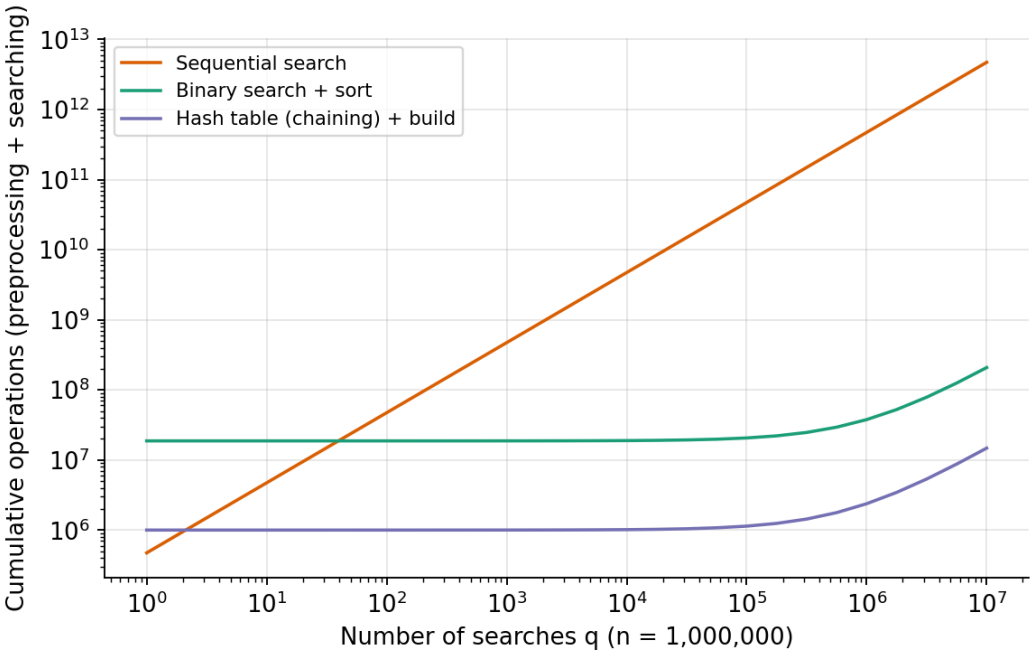


Figure 5: Cumulative cost (preprocessing plus searching) for n = 1,000,000 as the number of searches increases.

Table 16: Total operations for q searches on 1,000,000 records

q	Sequential	Binary (incl. sort)	Hash (incl. build)
1	500,000	18,716,503	1,000,001
10	5,000,005	18,716,674	1,000,014
100	50,000,050	18,718,379	1,000,138
10,000	5,000,005,000	18,905,988	1,013,763

12.2 Recommendation

For the original problem I recommend binary search on a sorted array as the standard method for the registration-number lookup, with sequential search retained only for a tiny or temporary list that is searched a handful of times. My reasons follow from the evidence.

- **Theoretical efficiency.** A search costs at most $\lfloor \log_2 n \rfloor + 1$ comparisons, a bound that holds for every input and does not depend on the data or on a hash function. It grows by one comparison for every doubling of the database.
- **Observed performance.** Searching a million records needs 19.0 comparisons and about 3.4 µs in my environment, compared with about 472,548 comparisons and 10,286 µs for sequential search.
- **Preprocessing cost.** Sorting a million records needed 18,716,484 comparisons, which is repaid after about 38 searches and is paid once. Subsequent changes can be handled by merging (Task 12).
- **Memory.** The index overhead is the sorted array itself, about 8.4 MB, against about 69 MB for the hash table.
- **Additional capability.** A sorted array supports range queries and ordered listing, such as all students of one admission batch whose registration numbers share a prefix. A hash table cannot do this.

12.3 Trade-offs and limitations

- Inserting a record into a sorted array requires shifting records or re-merging. This is acceptable only because insertions are infrequent.
- Hashing is faster per lookup, with about 1.4 comparisons against 19, but depends on the quality of the hash function. The adversarial test showed it can degrade to n comparisons, and it needs about eight times the memory plus periodic resizing as the database grows.
- Sequential search remains the better choice when the list is tiny or searched only once, because it needs no preparation. It stops being sensible as soon as the number of searches exceeds the break-even values above.

- The experiments were run in memory. For a disk-resident database each probe becomes a disk read, which favours methods with few probes and motivates tree-based indexes. The ranking of the three methods would stay the same.

Practicality as the database grows. The comparisons for binary search grow from 10 to 20 as the database grows a thousand-fold, and would be only about 24 for 16 million records. The solution therefore remains practical far beyond the sizes studied. The preprocessing grows as n log n, which is manageable because it is not repeated for every search.

13. Task 12 – Design Challenge: Re-evaluate and Adapt the Design

The new operating conditions are 50,000 searches per hour on 1,000,000 records, with new records added only once a day. I evaluated the design against this workload by running 50,000 queries (90% for existing keys, 10% for absent keys) on the same million-record dataset for binary search and hashing. Sequential search was measured on a sample of 100 queries and scaled to 50,000, because running it in full would take about 16 minutes. For the daily update I assumed 5,000 new records per day. This number is not given in the assignment, so it is my assumption.

A. Impact Analysis

What has changed. In the original problem the number of searches was unspecified, so the preprocessing cost had to be compared with an unknown amount of searching, and the records could have changed at any time. Now both quantities are known. There are 50,000 × 24 = 1,200,000 searches per day against one batch of updates, a ratio of about 240 searches for every record added (with my assumed 5,000 records per day).

Assumptions affected. Task 11 assumed that preprocessing could be needed whenever the data changed. That assumption no longer holds, because data change once per day.

Why search frequency is now important. The total cost of a design is the preprocessing cost plus q times the cost of one search. With q = 1.2 million per day, the term q × (cost per search) dominates, so a difference of even a few comparisons per search is multiplied 1.2 million times. A method with a linear per-search cost becomes a continuous load on the server.

Why the low update frequency matters. Preprocessing is done once and then shared by 1.2 million searches. That makes even a Θ(n log n) preparation negligible per search, and makes it affordable to use a structure that is expensive to update, such as a plain sorted array. It also removes the need for a dynamic structure that supports fast insertion, and the index can be rebuilt offline and swapped in at a quiet moment, so searches never see a half-updated index.

B. Evaluation of the Original Design

Table 17: Measured workload of 50,000 searches on 1,000,000 records

Option	Comparisons per hour	CPU time per hour	Fraction of one core
Sequential search	2.73e+10 (extrapolated)	987 s (extrapolated)	27.4%
Binary search	952,598	0.311 s	0.0086%
Hash table	65,495	0.040 s	0.0011%

Behaviour under 50,000 searches per hour. My original recommendation, binary search, needs 952,598 comparisons for the whole hour, i.e. 19.1 per search, and about 0.31 s of processor time out of 3,600 s. The arrival rate of about 14 searches per second (one every 72 ms) is thousands of times lower than the rate at which one core can serve binary searches. In contrast, sequential search would have needed about 27 billion comparisons, keeping a core about 27% busy and making each request wait 10 to 22 ms. Bursts of requests would queue up behind one another.

Preprocessing cost under the new conditions. If the sorted array were rebuilt by re-sorting everything each day, it would cost 18,800,058 comparisons (3.4 s in my environment). Spread over 1.2 million daily searches, that adds 15.7 comparisons to every search, nearly doubling the 19 comparisons the search itself needs. This is acceptable in absolute terms but wasteful.

Could another approach be better overall? Yes, in raw comparisons. The hash table needs only 1.31 comparisons per search, which is 14.5 times fewer than binary search, and was 7.7 times faster in the hour-long test. It also absorbs the daily batch by inserting only the new records (5,000 insertions). The absolute saving is small, however: about 0.27 s of processor time per hour.

C. Adaptation or Redesign

Decision: MODIFY the original solution. I retain binary search on a sorted array as the search strategy but change how the sorted array is maintained.

Revised strategy. Collect the day's new records, sort that small batch, and merge it into the existing sorted array with one linear pass. The merged array is built offline and swapped in atomically, so searches continue uninterrupted.

```
ALGORITHM DailyMerge(S, n, NewBatch, k)
INPUT:
    S – sorted array of n registration numbers
    NewBatch – array of k new registration numbers
OUTPUT:
    M – sorted array of n + k registration numbers
BEGIN
    MergeSort(NewBatch, 0, k - 1)
    i ← 0;  j ← 0;  t ← 0
    WHILE i < n AND j < k DO
        IF S[i] ≤ NewBatch[j] THEN  M[t] ← S[i];  i ← i + 1
        ELSE  M[t] ← NewBatch[j];  j ← j + 1
        END IF
        t ← t + 1
    END WHILE
    copy the remaining elements of S or NewBatch into M
    RETURN M
END
```

What has changed and the effect on complexity. The search remains Θ(log n). The daily preprocessing changes from re-sorting, Θ((n + k) log (n + k)), to a batch sort plus a merge, Θ(k log k + n + k), and since k is small compared with n this is effectively linear. Measured on the million-record array with 5,000 new records, the comparisons fell from 18,800,058 to 1,061,446 (17.7 times fewer), and the time from 3.40 s to 0.26 s. Per search, the amortised preprocessing is now 0.88 comparisons, under 5% of the cost of a search.

When I would redesign instead. The saving of a hash index over binary search is about 5.4 µs per search in my environment. To free even 10% of a core, the system would need about 18,458 searches per second, roughly 1,329 times the present load. Unless the load grows by orders of magnitude, or the response time requirement falls below what binary search provides, the hash index would add memory and a degradation risk for no practical benefit.

D. Evidence-Based Defence

Table 18: Comparison of the options under the changed requirement

Criterion	Sequential	Binary (modified)	Hash table
Search cost (n = 10^6)	Θ(n); about 545,000 comparisons	Θ(log n); about 19 comparisons	Θ(1) expected; about 1.3
Work per hour	2.7e+10	952,598	65,495
Daily preprocessing	None	1,061,446 comparisons (merge)	5,000 insertions
Index memory	None	about 8 MB	about 69 MB
Worst-case guarantee	n comparisons	20 comparisons	n if keys collide
Ordered and range queries	No	Yes	No

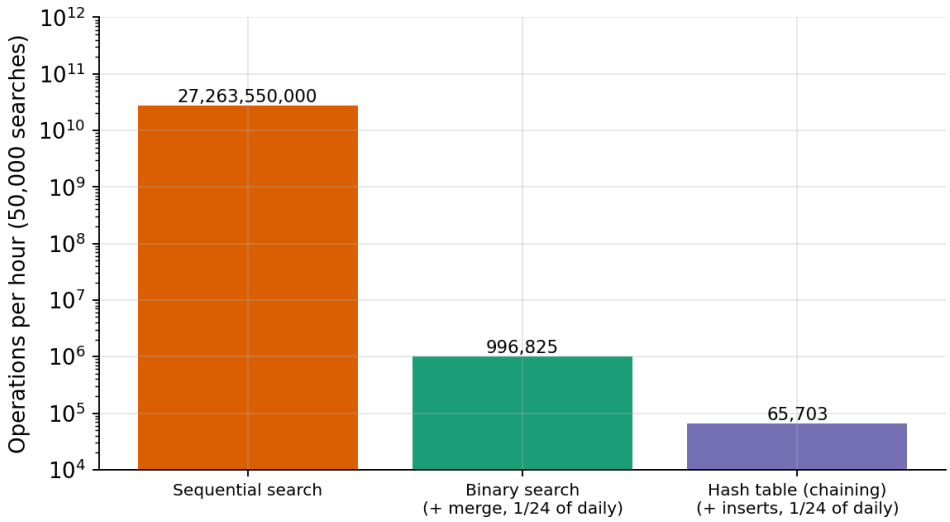


Figure 6: Operations per hour for the 50,000-search workload (logarithmic scale). The binary and hash bars include one twenty-fourth of the daily update cost.

The argument. First, the search frequency of 1.2 million per day makes sequential search unacceptable: its hourly work is four to five orders of magnitude larger than that of the other two, and every request would wait roughly 10 to 22 milliseconds. Second, the low update frequency makes preprocessing cheap per search, which supports the sorted-array design but also means the preprocessing method matters, so the full daily re-sort is replaced by a merge that costs 18 times fewer comparisons. Third, at this dataset size and request rate the extra speed of hashing saves a negligible amount of processor time while costing about eight times as much memory, giving up ordered queries and introducing a worst case of n comparisons. The decision follows from the numbers: modify, keep binary search, and replace the daily re-sort with a merge.

Sensitivity to scale. If the database grew tenfold to 10 million records, a binary search would need about 3 extra comparisons, sequential search would become ten times slower, and the daily merge would grow tenfold, remaining linear. The decision would still hold. If the request rate reached the threshold calculated above, the hash index should be reconsidered.

14. Conclusion

Three distinct strategies were designed, analysed mathematically and tested on databases from 1,000 to 1,000,000 records. The measurements agree with the derived cost functions: sequential search needs about n/2 comparisons on average, binary search needs about $\log_2 n - 1$, and a chained hash table needs about $1 + \alpha/2$. The recurrence $T(n) = T(\lfloor n/2 \rfloor) + 1$ was solved exactly as $\lfloor \log_2 n \rfloor + 1$, which matched the maximum number of probes observed at every size. Preprocessing pays for itself after a few dozen searches, so for a student-record system with frequent lookups, a sorted array with binary search is the recommended design. Under 50,000 searches per hour with daily updates, the design is kept but its maintenance is changed to a daily merge, which cuts the daily preprocessing by a factor of 18 and leaves the search itself unchanged.

Appendix A – Core Implementation

The three search routines are shown below. The complete instrumented library, the experiment driver, the raw results and the chart generator are listed in Appendix B.

```
def linear_search(A, key):
    comps = 0
    for i in range(len(A)):
        comps += 1
        if A[i] == key:
            return i, comps
    return -1, comps

def binary_recursive(A, low, high, key, stats):
    if low > high:
        return -1
    stats[0] += 1
    mid = (low + high) // 2
    if A[mid] == key:
        return mid
    elif key < A[mid]:
        return binary_recursive(A, low, mid - 1, key, stats)
    else:
        return binary_recursive(A, mid + 1, high, key, stats)

class ChainedHash:    # separate chaining, h(key) = key mod m
    def insert(self, key):
        b = key % self.m
        append key to self.buckets[b]
    def search(self, key):
        comps = 0
        for x in self.buckets[key % self.m]:
            comps += 1
            if x == key:  return True, comps
        return False, comps
```